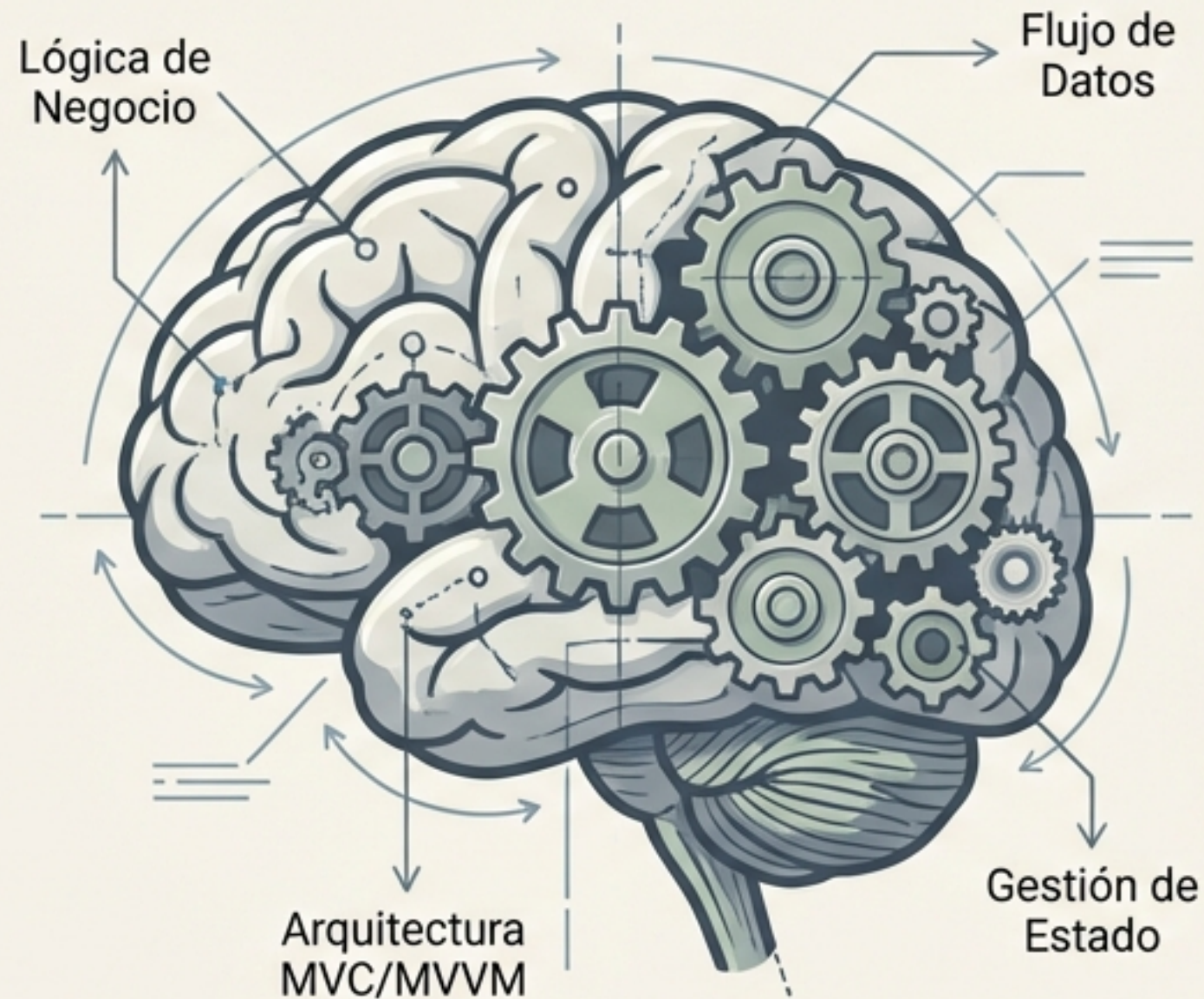


Anatomía de una App en Android

Construyendo el clásico Conecta 4 paso a paso.



La regla de oro del código moderno

Las mejores aplicaciones dividen sus responsabilidades en dos mundos completamente aislados.



Kotlin Puro

- El Cerebro (Lógica de negocio). Reglas, datos y cálculo.

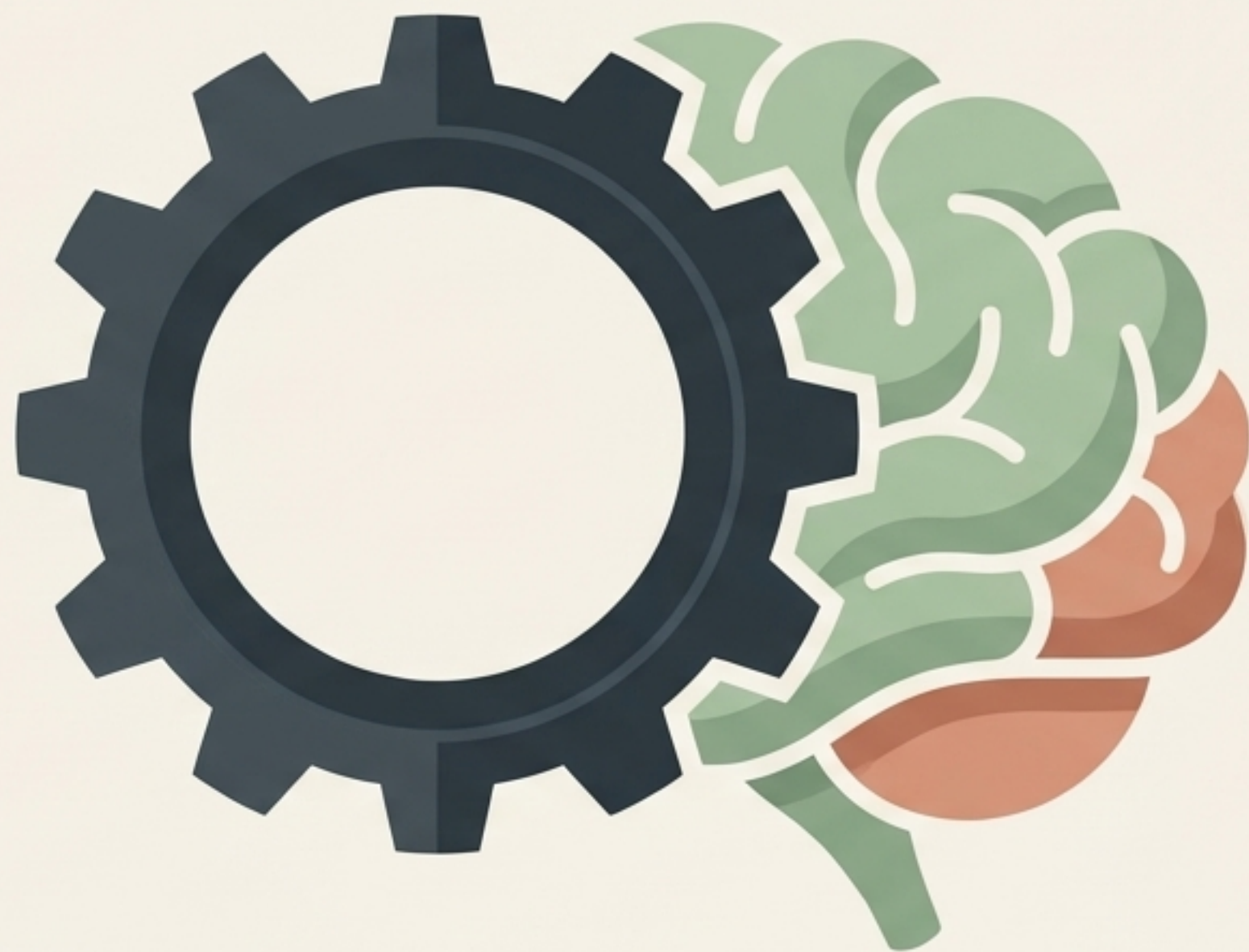


Jetpack Compose

- El Rostro (Interfaz de usuario). Colores, botones y pantalla.

Parte 1: Construyendo el Cerebro

Código Kotlin que puede vivir sin conocer nada sobre Android o pantallas.



Opciones cerradas para evitar errores

```
enum class Dificultad(val descripcion: String) {  
    FACIL("Fácil (Aleatorio)",  
    MEDIO("Medio (Defensivo)",  
    DIFICIL("Difícil (Inteligente)");  
}
```

Define un menú estricto de opciones que no se puede romper.

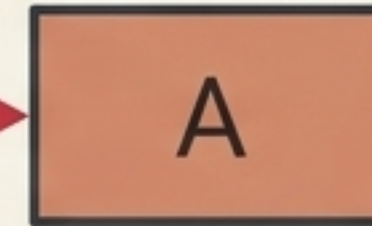
Cada nivel guarda un texto listo para mostrar en los botones.

Representando las fichas del tablero

Un truco de diseño: Se añaden espacios extra para compensar la anchura de los emojis. Así, el dibujo del tablero no se descuadra al imprimirlo en texto.

```
enum class Ficha(val simbolo: String) {  
    VACIO("  "),  
    JUGADOR("🔴"),  
    MAQUINA("🟡");  
}
```

Anchura carácter
estándar

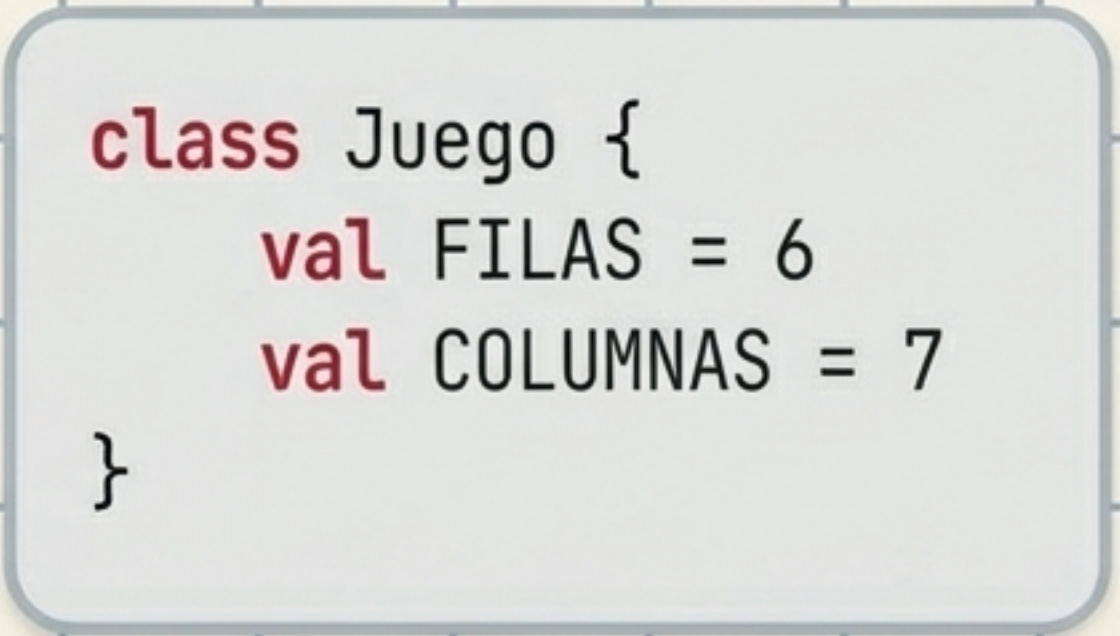


Anchura emoji
(más espacios)



El motor principal del juego

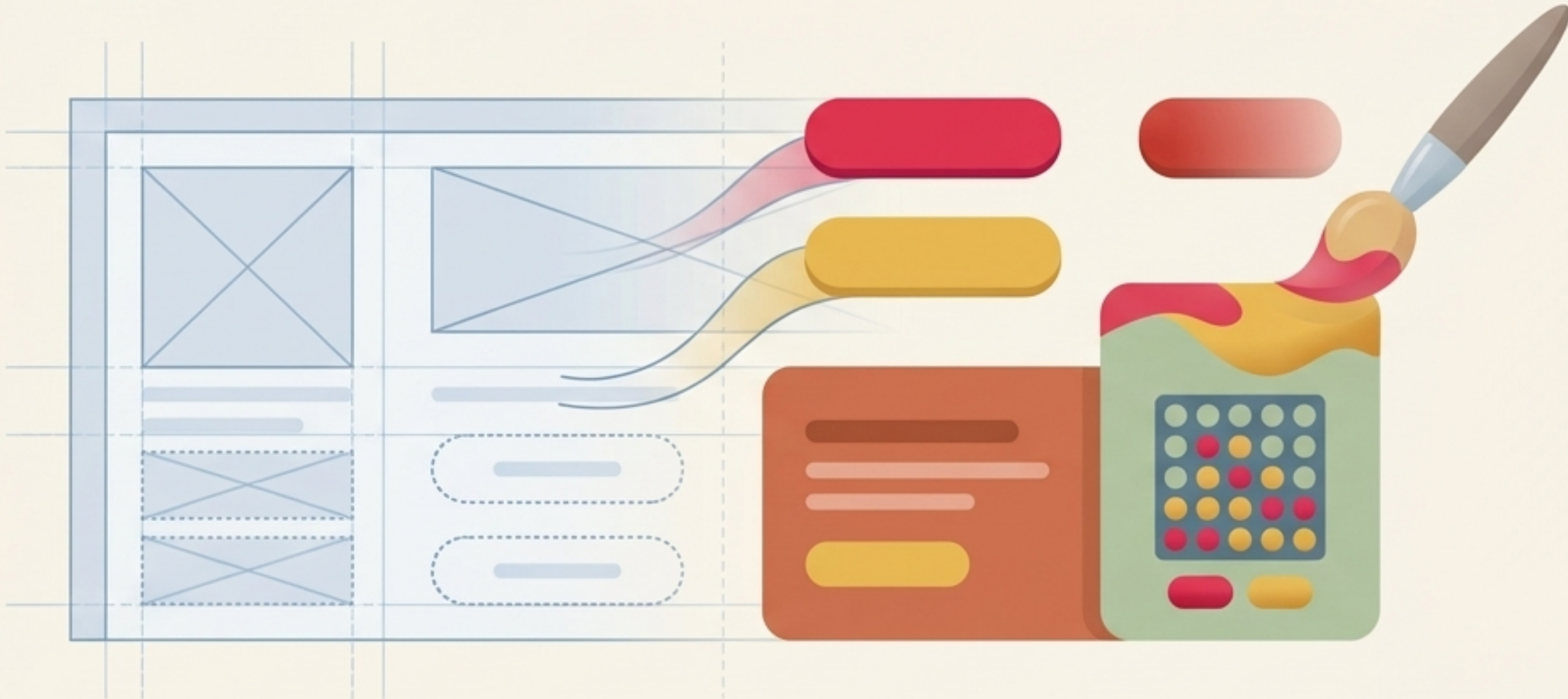
La clase principal guarda el tablero y las reglas. Es el núcleo absoluto de nuestra aplicación.



```
class Juego {  
    val FILAS = 6  
    val COLUMNAS = 7  
}
```

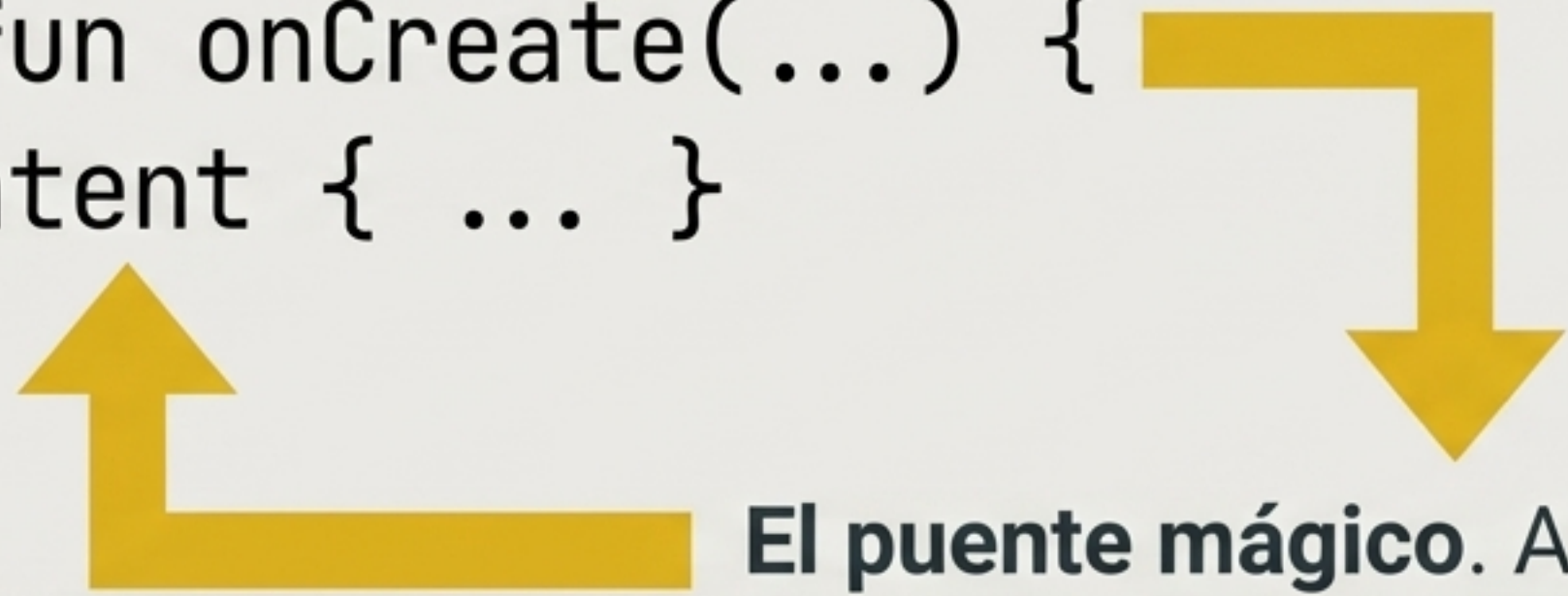

Parte 2: Diseñando el Rostro

Traduciendo nuestra lógica en una experiencia visual con Jetpack Compose.



El punto de entrada de la aplicación

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(...) {  
        setContent { ... }  
    }  
}
```

A diagram with two yellow arrows. One arrow starts at the 'setContent { ... }' line in the code and points down to the text 'El puente mágico...'. The other arrow starts at the 'setContent { ... }' line and points up to the 'setContent { ... }' line in the code.

El puente mágico. Aquí es donde Android deja de controlar y le cede el poder de dibujo a Jetpack Compose.

Preparando el lienzo de dibujo

`Modifier.fillMaxSize():`
Le indica a la superficie
que ocupe toda la pantalla.



Surface:

Actúa como el fondo
general de la app.

MaterialTheme:

Aplica los estilos del
sistema (colores y
tipografía).

La magia de la interfaz visual

La etiqueta `@Composable` transforma una función normal en un componente visual. Le avisa a Android que este bloqu de código sabe cómo **dibujar botones, textos o tableros** en la pantalla.

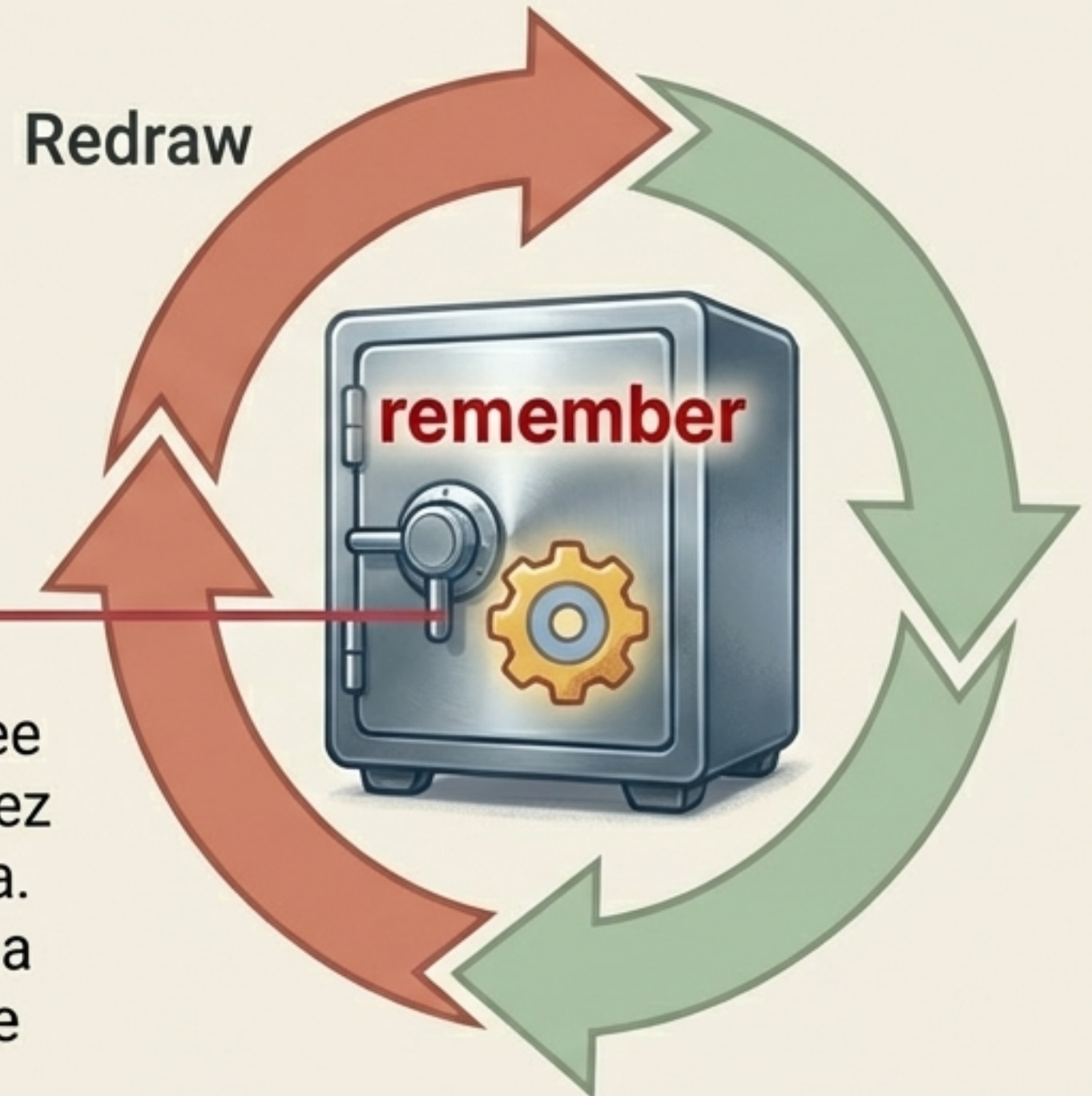


```
@Composable  
fun PantallaJuego() { ... }
```


Conectando el cerebro sin perder la memoria

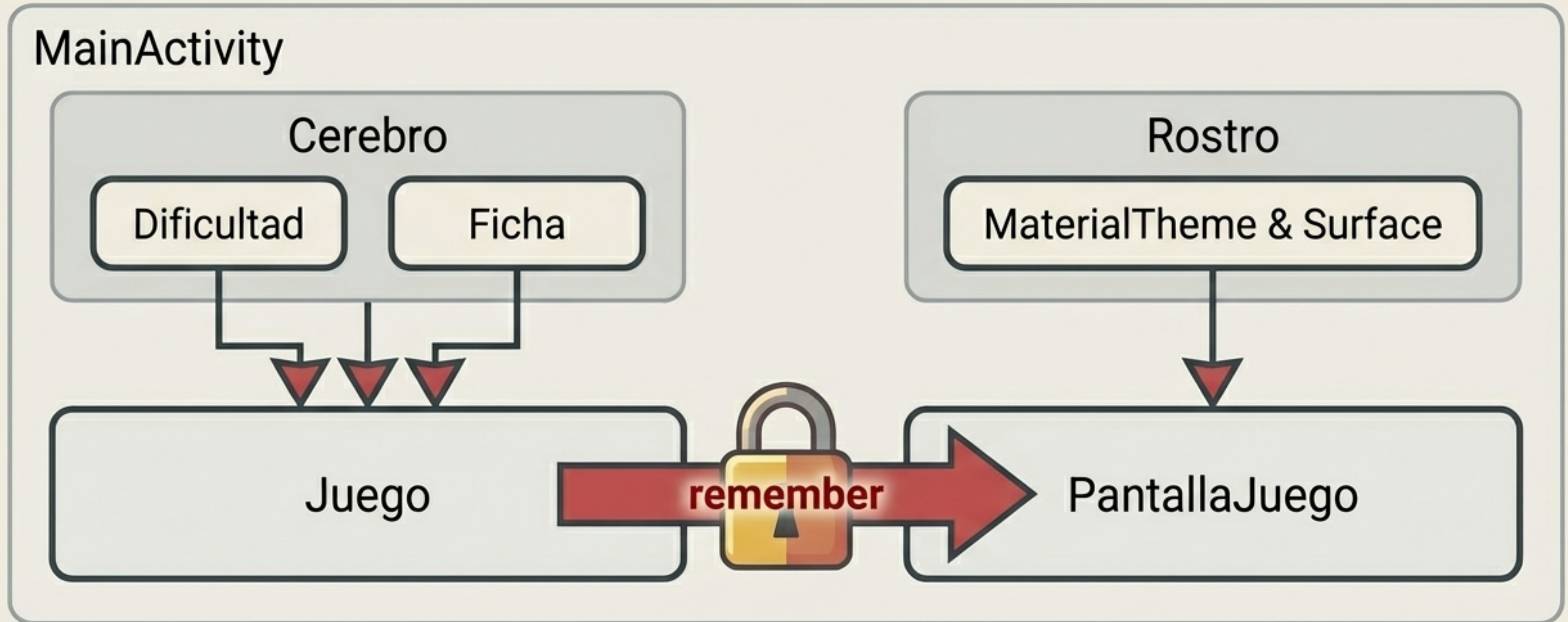
```
val motorJuego = remember {  
    Juego()  
}
```

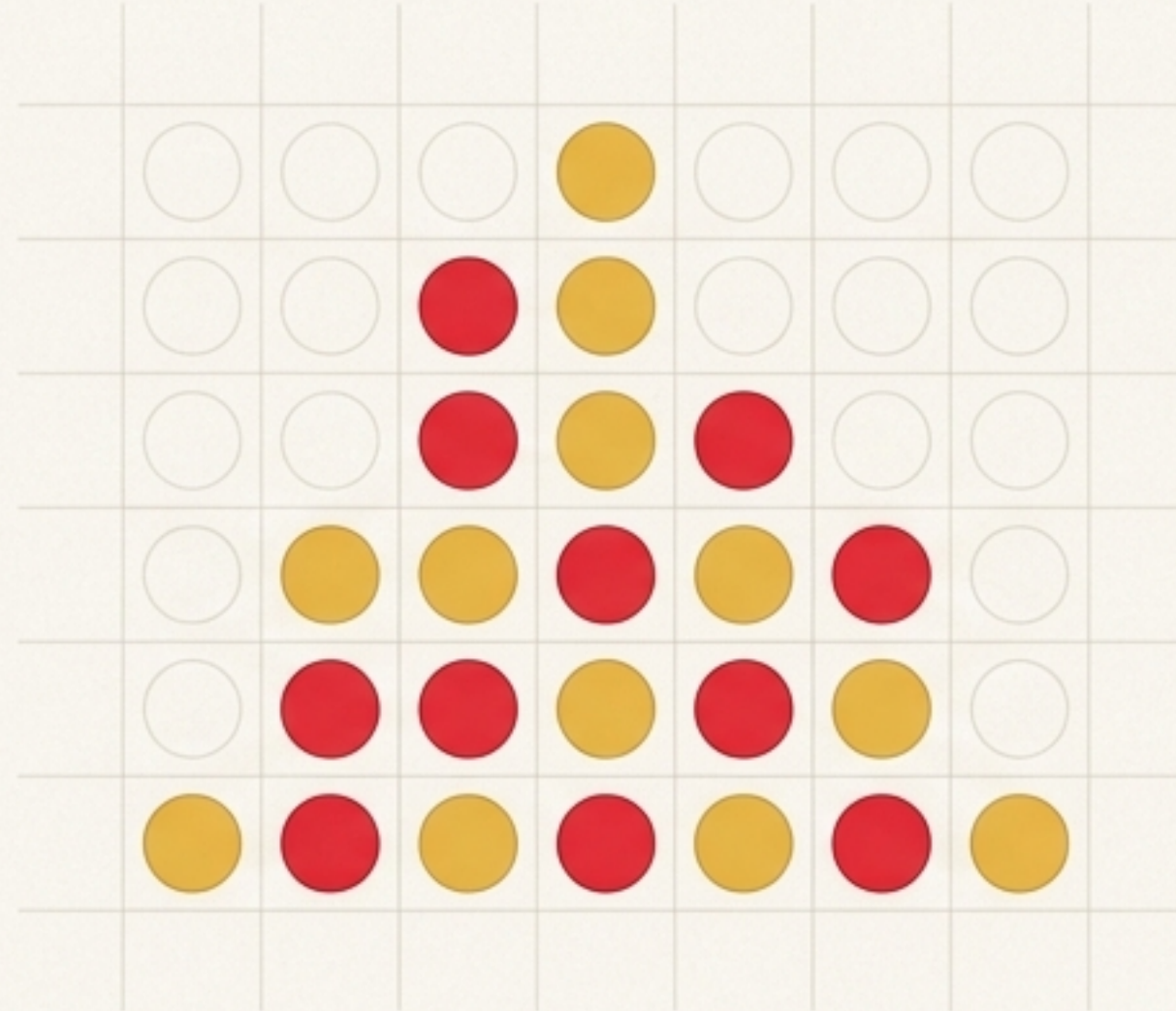
Evita que Compose cree un juego nuevo cada vez que redibuja la pantalla. Mantiene viva la partida durante todo el ciclo de la app.



El mapa completo de la arquitectura

Datos fijos estructuran la lógica. La lógica vive en la memoria. La memoria dibuja la pantalla.





La belleza de la separación

Al separar estrictamente el cálculo del juego de su representación visual, hemos creado un código escalable, fácil de leer y listo para evolucionar.